I want to allow people outside of my AWS account to access my Lambda resources

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether Lambda supports these features, see How AWS Lambda works with IAM.
- To learn how to provide access to your resources across AWS accounts that you own, see Providing access to an IAM user in another AWS account that you own in the *IAM User Guide*.
- To learn how to provide access to your resources to third-party AWS accounts, see Providing access to AWS accounts owned by third parties in the *IAM User Guide*.
- To learn how to provide access through identity federation, see Providing access to externally authenticated users (identity federation) in the *IAM User Guide*.
- To learn the difference between using roles and resource-based policies for cross-account access, see Cross account resource access in IAM in the *IAM User Guide*.

Create a governance strategy for Lambda functions and layers

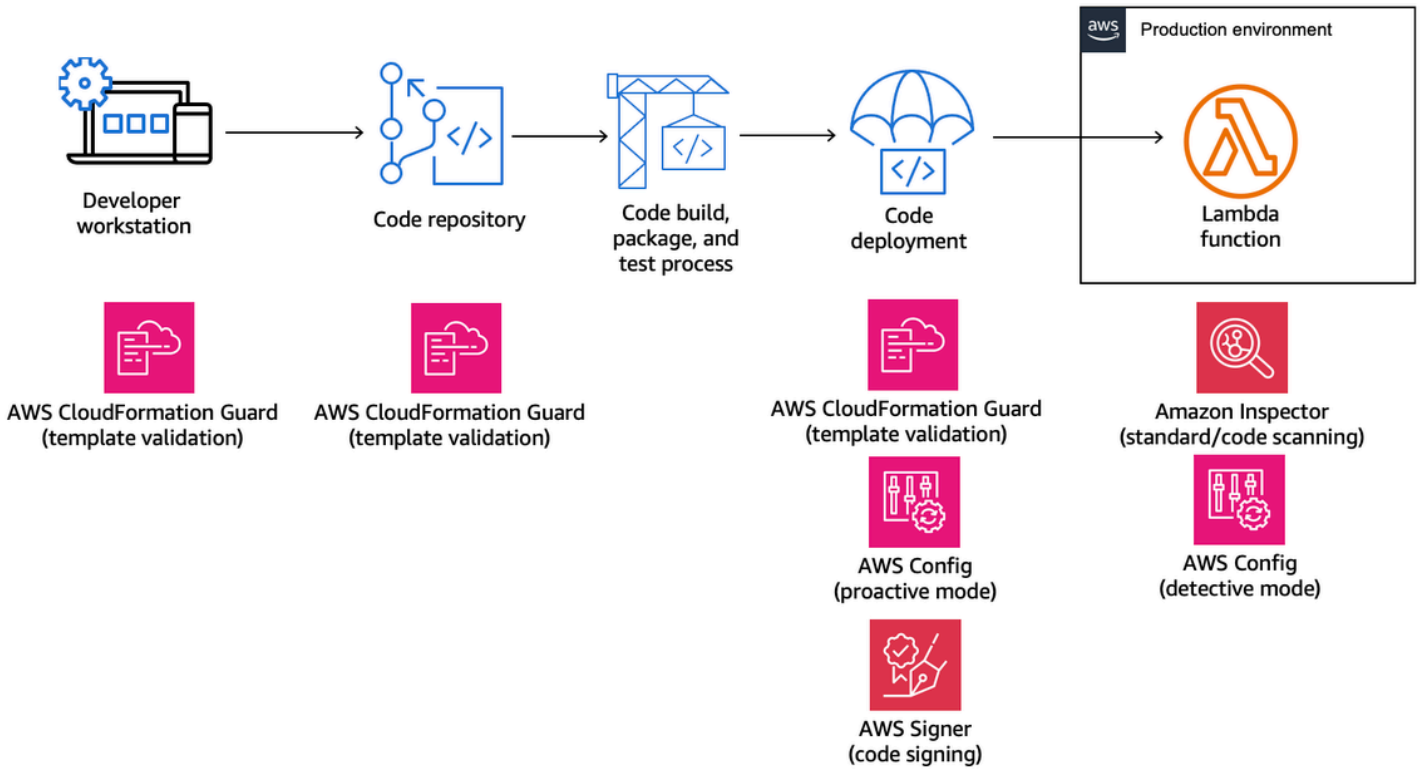
To build and deploy serverless, cloud-native applications, you must allow for agility and speed to market with appropriate governance and guardrails. You set business-level priorities, maybe emphasizing agility as the top priority, or alternatively emphasizing risk aversion via governance, guardrails, and controls. Realistically, you won't have an "either/or" strategy but an "and" strategy that balances both agility and guardrails in your software development lifecycle. No matter where these requirements fall in your company's lifecycle, governance capabilities are likely to become an implementation requirement in your processes and toolchains.

Here are a few examples of governance controls that an organization might implement for Lambda:

- Lambda functions must not be publicly accessible.
- Lambda functions must be attached to a VPC.
- Lambda functions should not use deprecated runtimes.
- Lambda functions must be tagged with a set of required tags.

- Lambda layers must not be accessible outside of the organization.
- Lambda functions with an attached security group must have matching tags between the function and security group.
- Lambda functions with an attached layer must use an approved version
- Lambda environment variables must be encrypted at rest with a customer managed key.

The following diagram is an example of an in-depth governance strategy that implements controls and policy throughout the software development and deployment process:



The following topics explain how to implement controls for developing and deploying Lambda functions in your organization, both for the startup and the enterprise. Your organization might already have tools in place. The following topics take a modular approach to these controls, so that you can pick and choose the components you actually need.

Topics

- Proactive controls for Lambda with AWS CloudFormation Guard
- Implement preventative controls for Lambda with AWS Config
- Detect non-compliant Lambda deployments and configurations with AWS Config

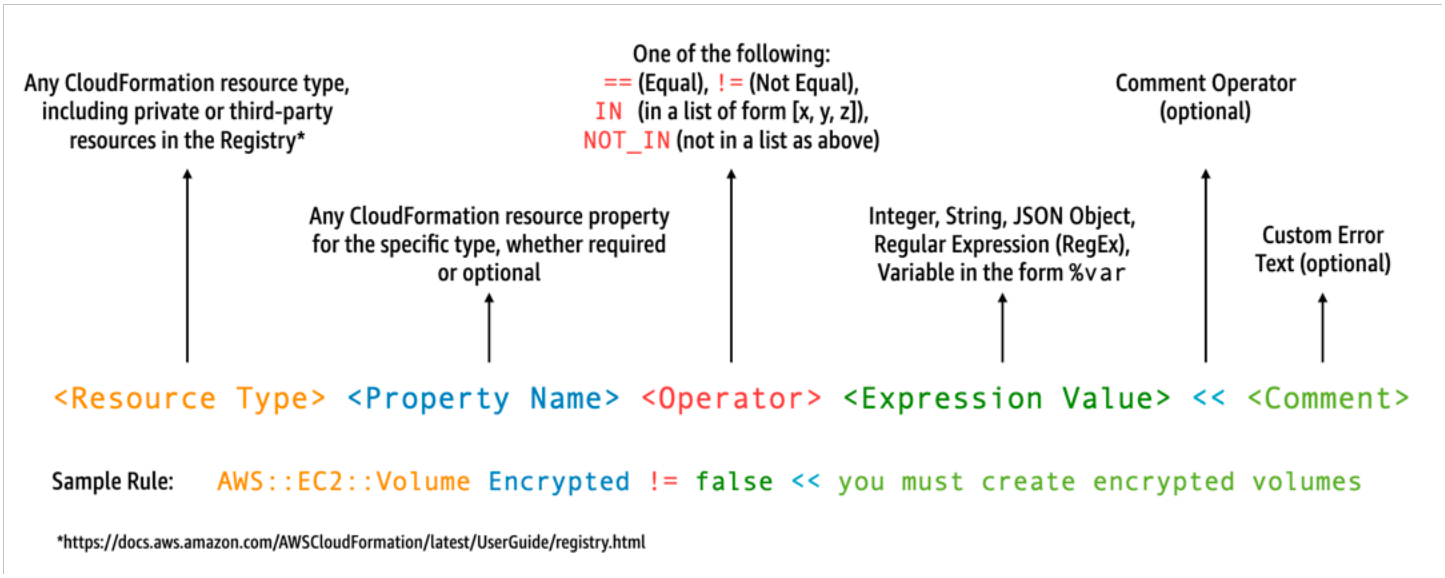
- Lambda code signing with AWS Signer
- Automate security assessments for Lambda with Amazon Inspector
- Implement observability for Lambda security and compliance

Proactive controls for Lambda with AWS CloudFormation Guard

AWS CloudFormation Guard is an open-source, general-purpose, policy-as-code evaluation tool. This can be used for preventative governance and compliance by validating Infrastructure as Code (IaC) templates and service compositions against policy rules. These rules can be customized based on your team or organizational requirements. For Lambda functions, the Guard rules can be used to control resource creation and configuration updates by defining the required property settings needed while creating or updating a Lambda function.

Compliance administrators define the list of controls and governance policies that are required for deploying and updating Lambda functions. Platform administrators implement the controls in CI/CD pipelines, as pre-commit validation webhooks with code repositories, and provide developers with command line tools for validating templates and code on local workstations. Developers author code, validate templates with command line tools, and then commit code to repositories, which are then automatically validated via the CI/CD pipelines prior to deployment into an AWS environment.

Guard allows you to write your rules and implement your controls with a domain-specific language as follows.



For example, suppose you want to ensure that developers choose only the latest runtimes. You could specify two different policies, one to identify runtimes that are already deprecated and another to identify runtimes that are to be deprecated soon. To do this, you might write the following `etc/rules.guard` file:

```
let lambda_functions = Resources.*[
    Type == "AWS::Lambda::Function"
]

rule lambda_already_deprecated_runtime when %lambda_functions !empty {
    %lambda_functions {
        Properties {
            when Runtime exists {
                Runtime !in ["dotnetcore3.1", "nodejs12.x", "python3.6", "python2.7",
 "dotnet5.0", "dotnetcore2.1", "ruby2.5", "nodejs10.x", "nodejs8.10", "nodejs4.3",
 "nodejs6.10", "dotnetcore1.0", "dotnetcore2.0", "nodejs4.3-edge", "nodejs"] <<Lambda
 function is using a deprecated runtime.>>
            }
        }
    }
}

rule lambda_soon_to_be_deprecated_runtime when %lambda_functions !empty {
    %lambda_functions {
        Properties {
            when Runtime exists {
                Runtime !in ["nodejs16.x", "nodejs14.x", "python3.7", "java8",
 "dotnet7", "go1.x", "ruby2.7", "provided"] <<Lambda function is using a runtime that
 is targeted for deprecation.>>
            }
        }
    }
}
```

Now suppose you write the following `iac/lambda.yaml` CloudFormation template that defines a Lambda function:

```
  Fn:
    Type: AWS::Lambda::Function
    Properties:
      Runtime: python3.7
      CodeUri: src
      Handler: fn.handler
      Role: !GetAtt FnRole.Arn
      Layers:
        - arn:aws:lambda:us-east-1:111122223333:layer:LambdaInsightsExtension:35
```

After installing the Guard utility, validate your template:

```
cfn-guard validate --rules etc/rules.guard --data iac/lambda.yaml
```

The output looks like this:

```
lambda.yaml Status = FAIL
FAILED rules
rules.guard/lambda_soon_to_be_deprecated_runtime
---
Evaluating data lambda.yaml against rules rules.guard
Number of non-compliant resources 1
Resource = Fn {
  Type      = AWS::Lambda::Function
  Rule = lambda_soon_to_be_deprecated_runtime {
    ALL {
      Check =  Runtime not IN
 ["nodejs16.x","nodejs14.x","python3.7","java8","dotnet7","go1.x","ruby2.7","provided"]
 {
        ComparisonError {
          Message          = Lambda function is using a runtime that is targeted for
 deprecation.
          Error            = Check was not compliant as property [/Resources/
Fn/Properties/Runtime[L:88,C:15]] was not present in [(resolved, Path=[L:0,C:0]
 Value=["nodejs16.x","nodejs14.x","python3.7","java8","dotnet7","go1.x","ruby2.7","provided"])]
        }
          PropertyPath    = /Resources/Fn/Properties/Runtime[L:88,C:15]
          Operator        = NOT IN
          Value           = "python3.7"
          ComparedWith    =
 [["nodejs16.x","nodejs14.x","python3.7","java8","dotnet7","go1.x","ruby2.7","provided"]]
          Code:
               86.  Fn:
               87.    Type: AWS::Lambda::Function
               88.    Properties:
               89.      Runtime: python3.7
               90.      CodeUri: src
               91.      Handler: fn.handler


      }
    }
  }
}
```

Guard allows your developers to see from their local developer workstations that they need to update the template to use a runtime that is allowed by the organization. This happens prior to committing to a code repository and subsequently failing checks within a CI/CD pipeline. As a result, your developers get this feedback on how to develop compliant templates and shift their time to writing code that delivers business value. This control can be applied on the local developer workstation, in a pre-commit validation webhook, and/or in the CI/CD pipeline prior to deployment.

Caveats

If you're using AWS Serverless Application Model (AWS SAM) templates to define Lambda functions, be aware that you need to update the Guard rule to search for the `AWS::Serverless::Function` resource type as follows.

```
let lambda_functions = Resources.*[
    Type == "AWS::Serverless::Function"
]
```

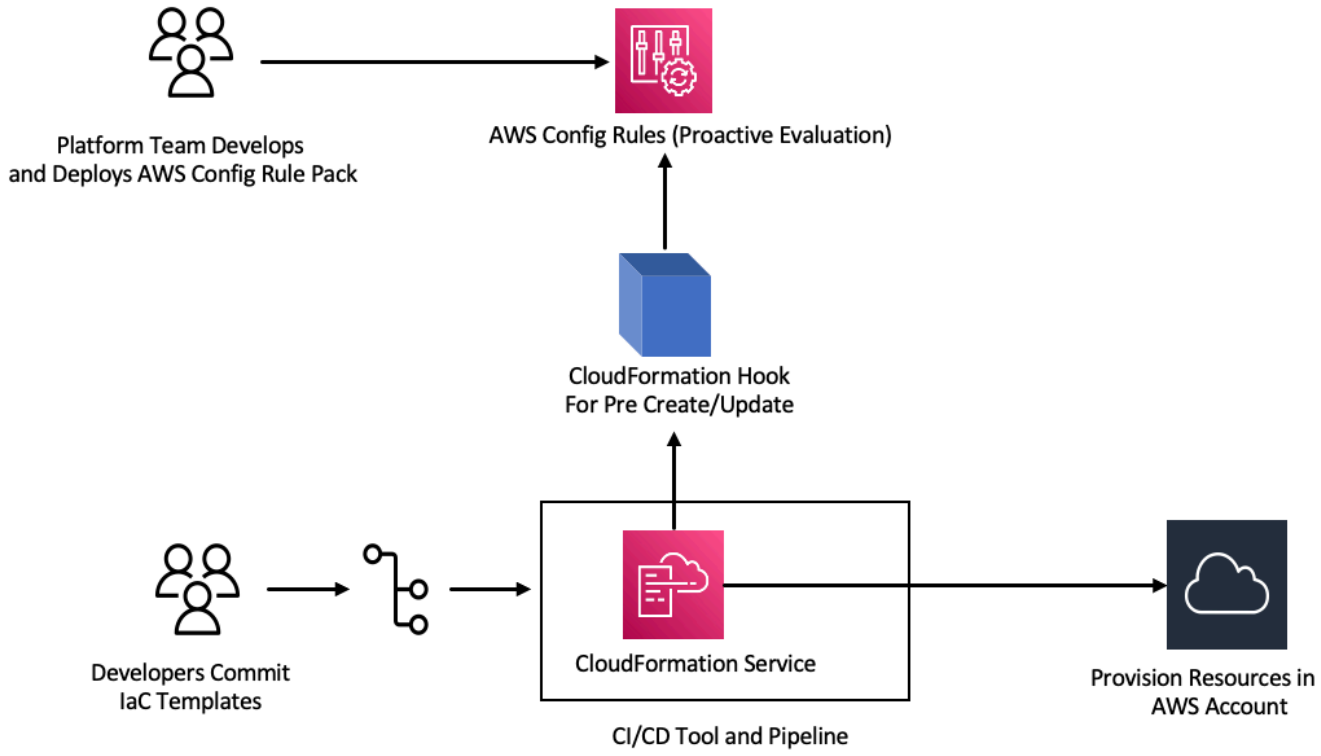
Guard also expects the properties to be included within the resource definition. Meanwhile, AWS SAM templates allow for properties to be specified in a separate Globals section. Properties that are defined in the Globals section are not validated with your Guard rules.

As outlined in the Guard troubleshooting documentation, be aware that Guard doesn't support short-form intrinsics like `!GetAtt` or `!Sub` and instead requires using the expanded forms: `Fn::GetAtt` and `Fn::Sub`. (The earlier example doesn't evaluate the Role property, so the short-form intrinsic was used for simplicity.)

Implement preventative controls for Lambda with AWS Config

It is essential to ensure compliance in your serverless applications as early in the development process as possible. In this topic, we cover how to implement preventative controls using AWS Config. This allows you to implement compliance checks earlier in the development process and enables you to implement the same controls in your CI/CD pipelines. This also standardizes your controls in a centrally managed repository of rules so that you can apply your controls consistently across your AWS accounts.

For example, suppose your compliance administrators defined a requirement to ensure that all Lambda functions include AWS X-Ray tracing. With AWS Config's proactive mode, you can run compliance checks on your Lambda function resources before deployment, reducing the risk of deploying improperly configured Lambda functions and saving developers time by giving them faster feedback on infrastructure as code templates. The following is a visualization of the flow for preventative controls with AWS Config:



Consider a requirement that all Lambda functions must have tracing enabled. In response, the platform team identifies the need for a specific AWS Config rule to run proactively across all

accounts. This rule flags any Lambda function that lacks a configured X-Ray tracing configuration as a non-compliant resource. The team develops a rule, packages it in a conformance pack, and deploys the conformance pack across all AWS accounts to ensure that all accounts in the organization uniformly apply these controls. You can write the rule in AWS CloudFormation Guard 2.x.x syntax, which takes the following form:

```
rule name when condition { assertion }
```

The following is a sample Guard rule that checks to ensure Lambda functions has tracing enabled:

```
rule lambda_tracing_check {
  when configuration.tracingConfig exists {
      configuration.tracingConfig.mode == "Active"
  }
}
```

The platform team takes further action by mandating that every AWS CloudFormation deployment invokes a pre-create/update hook. They assume full responsibility for developing this hook and configuring the pipeline, strengthening the centralized control of compliance rules and sustaining their consistent application across all deployments. To develop, package, and register a hook, see Developing AWS CloudFormation Hooks in the CloudFormation Command Line Interface (CFN-CLI) documentation. You can use the CloudFormation CLI to create the hook project:

```
cfn init
```

This command asks you for some basic information about your hook project and creates a project with following files in it:

```
README.md
<hook-name>.json
rpdk.log
src/handler.py
template.yml
hook-role.yaml
```

As a hook developer, you need to add the desired target resource type in the `<hook-name>.json` configuration file. In the configuration below, a hook is configured to execute before any Lambda function is created using CloudFormation. You can add similar handlers for `preUpdate` and `preDelete` actions as well.

```
    "handlers": {
        "preCreate": {
            "targetNames": [
                "AWS::Lambda::Function"
            ],
            "permissions": []
        }
    }
```

You also need to ensure that the CloudFormation hook has appropriate permissions to call the AWS Config APIs. You can do that by updating the role definition file named `hook-role.yaml`. The role definition file has the following trust policy by default, which allows CloudFormation to assume the role.

```
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          - Effect: Allow
            Principal:
              Service:
                - hooks.cloudformation.amazonaws.com
                - resources.cloudformation.amazonaws.com
```

To allow this hook to call config APIs, you must add following permissions to the Policy statement. Then you submit the hook project using the `cfn submit` command, where CloudFormation creates a role for you with the required permissions.

```
      Policies:
        - PolicyName: HookTypePolicy
          PolicyDocument:
            Version: '2012-10-17'
            Statement:
              - Effect: Allow
                Action:
                  - "config:Describe*"
                  - "config:Get*"
                  - "config:List*"
                  - "config:SelectResourceConfig"
                Resource: "*
```

Next, you need to write a Lambda function in a `src/handler.py` file. Within this file, you find methods named `preCreate`, `preUpdate`, and `preDelete` already created when you initiated the project. You aim to write a common, reusable function that calls the AWS Config `StartResourceEvaluation` API in proactive mode using the AWS SDK for Python (Boto3). This API call takes resource properties as input and evaluates the resource against the rule definition.

```
def validate_lambda_tracing_config(resource_type, function_properties:
 MutableMapping[str, Any]) -> ProgressEvent:
  LOG.info("Fetching proactive data")
  config_client = boto3.client('config')
  resource_specs = {
      'ResourceId': 'MyFunction',
      'ResourceType': resource_type,
      'ResourceConfiguration': json.dumps(function_properties),
      'ResourceConfigurationSchemaType': 'CFN_RESOURCE_SCHEMA'
  }
  LOG.info("Resource Specifications:", resource_specs)
  eval_response = config_client.start_resource_evaluation(EvaluationMode='PROACTIVE',
 ResourceDetails=resource_specs, EvaluationTimeout=60)
  ResourceEvaluationId = eval_response.ResourceEvaluationId
  compliance_response =
 config_client.get_compliance_details_by_resource(ResourceEvaluationId=ResourceEvaluationId)
  LOG.info("Compliance Verification:",
 compliance_response.EvaluationResults[0].ComplianceType)
  if "NON_COMPLIANT" == compliance_response.EvaluationResults[0].ComplianceType:
      return ProgressEvent(status=OperationStatus.FAILED, message="Lambda function
 found with no tracing enabled : FAILED", errorCode=HandlerErrorCode.NonCompliant)
  else:
      return ProgressEvent(status=OperationStatus.SUCCESS, message="Lambda function
 found with tracing enabled : PASS.")
```

Now you can call the common function from the handler for the pre-create hook. Here's an example of the handler:

```
@hook.handler(HookInvocationPoint.CREATE_PRE_PROVISION)
def pre_create_handler(
        session: Optional[SessionProxy],
        request: HookHandlerRequest,
        callback_context: MutableMapping[str, Any],
        type_configuration: TypeConfigurationModel
) -> ProgressEvent:
    LOG.info("Starting execution of the hook")
```

```
    target_name = request.hookContext.targetName
    LOG.info("Target Name:", target_name)
    if "AWS::Lambda::Function" == target_name:
        return validate_lambda_tracing_config(target_name,
            request.hookContext.targetModel.get("resourceProperties")
        )
    else:
        raise exceptions.InvalidRequest(f"Unknown target type: {target_name}")
```

After this step you can register the hook and configure it to listen to all AWS Lambda function creation events.

A developer prepares the infrastructure as code (IaC) template for a serverless microservice using Lambda. This preparation includes adherence to internal standards, followed by locally testing and committing the template to the repository. Here's an example IaC template:

```
  MyLambdaFunction:
  Type: 'AWS::Lambda::Function'
  Properties:
    Handler: index.handler
    Role: !GetAtt LambdaExecutionRole.Arn
    FunctionName: MyLambdaFunction
    Code:
      ZipFile: |
        import json

        def handler(event, context):
            return {
                'statusCode': 200,
                'body': json.dumps('Hello World!')
            }
    Runtime: python3.14
    TracingConfig:
        Mode: PassThrough
    MemorySize: 256
    Timeout: 10
```

As part of the CI/CD process, when the CloudFormation template is deployed, the CloudFormation service invokes the pre-create/update hook right before provisioning `AWS::Lambda::Function` resource type. The hook utilizes AWS Config rules running in proactive mode to verify that the Lambda function configuration includes the mandated tracing configuration. The response from the hook determines the next step. If compliant, the hook signals success, and CloudFormation

proceeds to provision the resources. If not, the CloudFormation stack deployment fails, the pipeline comes to an immediate halt, and the system records the details for subsequent review. Compliance notifications are sent to the relevant stakeholders.

You can find the hook success/fail information in the CloudFormation console:

Stack info | Events | Resources | Outputs | Parameters | Template | Change sets

Events (19)

Search events

Timestamp	Logical ID	Status	Status reason	Hook invocations
2023-08-29 23:50:23 UTC-0500	HookTestStack	ROLLBACK_COMPLETE	-	-
2023-08-29 23:50:22 UTC-0500	LambdaExecutionRole	DELETE_COMPLETE	-	-
2023-08-29 23:50:21 UTC-0500	MyApi	DELETE_COMPLETE	-	-
2023-08-29 23:50:20 UTC-0500	LambdaExecutionRole	DELETE_IN_PROGRESS	-	-
2023-08-29 23:50:20 UTC-0500	MyLambdaFunction	DELETE_COMPLETE	-	-
2023-08-29 23:50:20 UTC-0500	MyApi	DELETE_IN_PROGRESS	-	-
2023-08-29 23:50:18 UTC-0500	HookTestStack	ROLLBACK_IN_PROGRESS	The following resource(s) failed to create: [MyLambdaFunction]. Rollback requested by user.	-
2023-08-29 23:50:17 UTC-0500	MyLambdaFunction	CREATE_FAILED	The following hook(s) failed: [AWSSamples::LambdaTracingCheck::Hook]	-
2023-08-29 23:50:17 UTC-0500	MyLambdaFunction	CREATE_IN_PROGRESS	-	AWSSamples::LambdaTracingCheck::Hook
2023-08-29 23:50:16 UTC-0500	MyLambdaFunction	CREATE_IN_PROGRESS	-	AWSSamples::LambdaTracingCheck::Hook
2023-08-29 23:50:15 UTC-0500	MyLambdaFunction	CREATE_IN_PROGRESS	-	-
2023-08-29 23:50:14 UTC-0500	LambdaExecutionRole	CREATE_COMPLETE	-	-
2023-08-29 23:49:59 UTC-0500	MyApi	CREATE_COMPLETE	-	-
2023-08-29 23:49:59 UTC-0500	MyApi	CREATE_IN_PROGRESS	Resource creation Initiated	-
2023-08-29 23:49:58 UTC-0500	LambdaExecutionRole	CREATE_IN_PROGRESS	Resource creation Initiated	-
2023-08-29 23:49:58 UTC-0500	LambdaExecutionRole	CREATE_IN_PROGRESS	-	-
2023-08-29 23:49:58 UTC-0500	MyApi	CREATE_IN_PROGRESS	-	-
2023-08-29 23:49:55 UTC-0500	HookTestStack	CREATE_IN_PROGRESS	User Initiated	-
2023-08-29 23:49:50 UTC-0500	HookTestStack	REVIEW_IN_PROGRESS	User Initiated	-

If you have logs enabled for your CloudFormation hook, you can capture the hook evaluation result. Here is a sample log for a hook with a failed status, indicating that the Lambda function does not have X-Ray enabled:

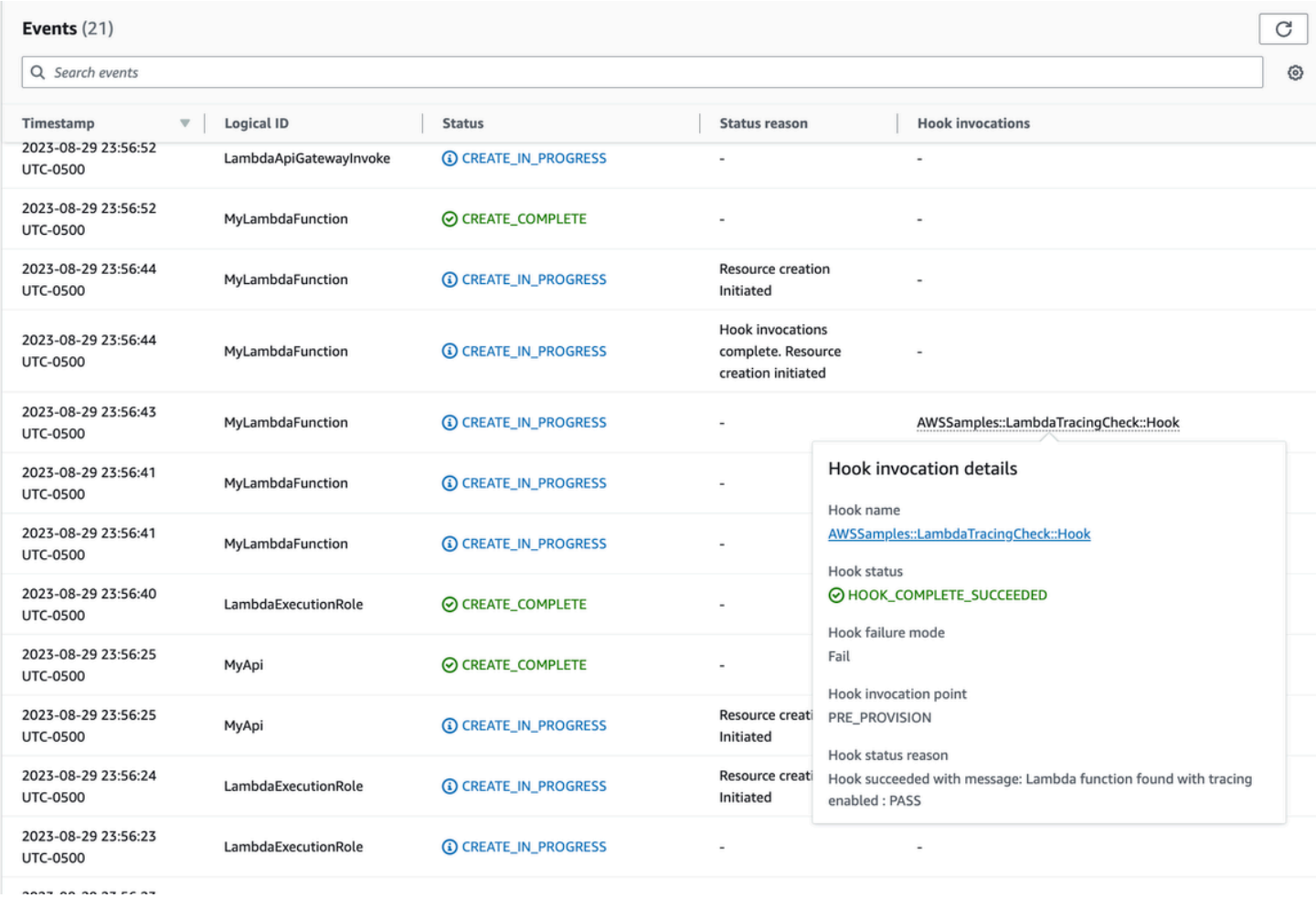
```
2023-08-29T23:50:17.574-05:00    ProgressEvent(status=<OperationStatus.FAILED: 'FAILED'>, errorCode=<HandlerErrorCode.NonCompliant: 'NonCompliant'…

ProgressEvent(status=<OperationStatus.FAILED: 'FAILED'>, errorCode=<HandlerErrorCode.NonCompliant: 'NonCompliant'>, message='Lambda
function found with no tracing enabled : FAILED', result=None, callbackContext=None, callbackDelaySeconds=0, resourceModel=None,
resourceModels=None, nextToken=None)
```

Copy

No newer events at this moment. *Auto retry paused.* *Resume*

If the developer chooses to change the IaC to update `TracingConfig Mode` value to `Active` and redeploy, the hook executes successfully and the stack proceeds with creating the Lambda resource.



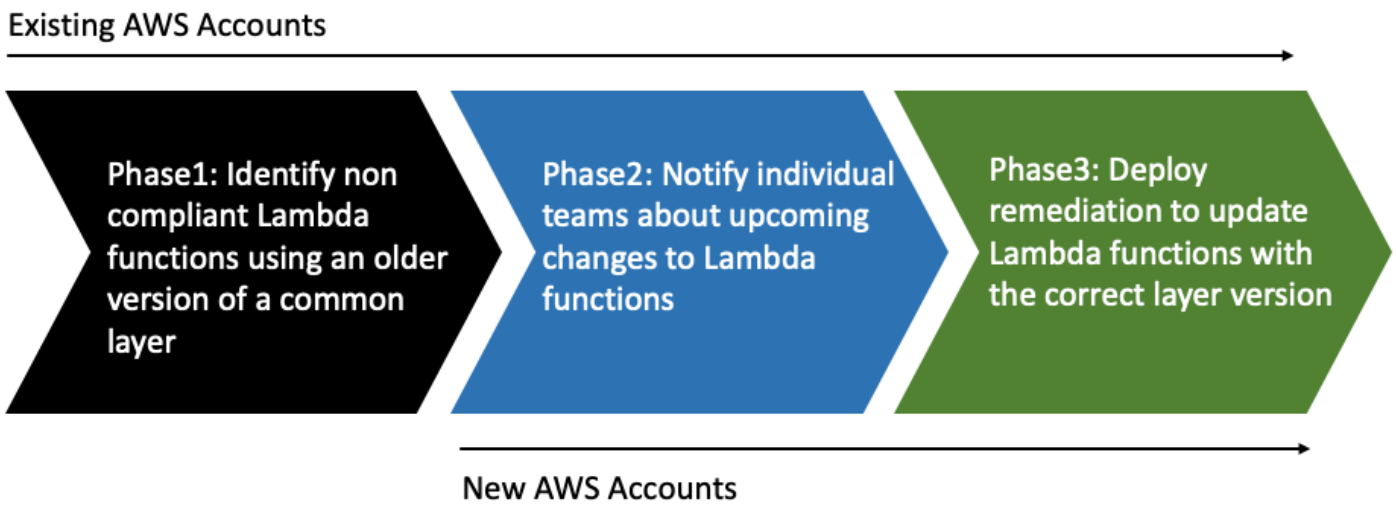
In this way, you can implement preventative controls with AWS Config in proactive mode when developing and deploying serverless resources in your AWS accounts. By integrating AWS Config rules into the CI/CD pipeline, you can identify and optionally block non-compliant resource deployments, such as Lambda functions that lack an active tracing configuration. This ensures that only resources that comply with the latest governance policies are deployed into your AWS environments.

Detect non-compliant Lambda deployments and configurations with AWS Config

In addition to proactive evaluation, AWS Config can also reactively detect resource deployments and configurations that do not comply with your governance policies. This is important because governance policies evolve as your organization learns and implements new best practices.

Consider a scenario where you set a brand new policy when deploying or updating Lambda functions: All Lambda functions must always use a specific, approved Lambda layer version. You can configure AWS Config to monitor new or updated functions for layer configurations. If AWS Config detects a function that is not using an approved layer version, it flags the function as a non-compliant resource. You can optionally configure AWS Config to automatically remediate the resource by specifying a remediation action using an AWS Systems Manager automation document. For example, you could write an automation document in Python using the AWS SDK for Python (Boto3), which updates the non-compliant function to point to the approved layer version. Thus, AWS Config serves as both a detective and corrective control, automating compliance management.

Let's break down this process into three important implementation phases:



Phase 1: Identify access resources

Start by activating AWS Config across your accounts and configuring it to record AWS Lambda functions. This allows AWS Config to observe when Lambda functions are created or updated. You can then configure custom policy rules to check for specific policy violations, which use AWS CloudFormation Guard syntax. Guard rules take the following general form:

```
rule name when condition { assertion }
```

Below is a sample rule that checks to ensure that a layer is not set to an old layer version:

```
rule desiredlayer when configuration.layers !empty {
    some configuration.layers[*].arn != CONFIG_RULE_PARAMETERS.OldLayerArn
}
```

Let's understand the rule syntax and structure:

- **Rule name:** The name of the rule in the provided example is `desiredlayer`.
- **Condition:** This clause specifies the condition under which the rule should be checked. In the provided example, the condition is `configuration.layers !empty`. This means the resource should be evaluated only when the `layers` property in the configuration isn't empty.
- **Assertion:** After the `when` clause, an assertion determines what the rule checks. The assertion `some configuration.layers[*].arn != CONFIG_RULE_PARAMETERS.OldLayerArn` checks if any of the Lambda layer ARNs do not match the `OldLayerArn` value. If they do not match, the assertion is true and the rule passes; otherwise, it fails.

`CONFIG_RULE_PARAMETERS` is a special set of parameters that is configured with the AWS Config rule. In this case, `OldLayerArn` is a parameter inside `CONFIG_RULE_PARAMETERS`. This allows users to provide a specific ARN value that they consider old or deprecated, and then the rule checks if any Lambda functions are using this old ARN.

Phase 2: Visualize and design

AWS Config gathers configuration data and stores that data in Amazon Simple Storage Service (Amazon S3) buckets. You can use Amazon Athena to query this data directly from your S3 buckets. With Athena, you can aggregate this data at the organizational level, generating a holistic view of your resource configurations across all your accounts. To set up aggregation of resource configuration data, see Visualizing AWS Config data using Athena and Amazon Quick on the AWS Cloud Operations and Management blog.

The following is a sample Athena query to identify all Lambda functions using a particular layer ARN:

```
WITH unnested AS (
```

```
    SELECT
      item.awsaccountid AS account_id,
      item.awsregion AS region,
      item.configuration AS lambda_configuration,
      item.resourceid AS resourceid,
      item.resourcename AS resourcename,
      item.configuration AS configuration,
      json_parse(item.configuration) AS lambda_json
    FROM
      default.aws_config_configuration_snapshot,
      UNNEST(configurationitems) as t(item)
    WHERE
      "dt" = 'latest'
      AND item.resourcetype = 'AWS::Lambda::Function'
  )

  SELECT DISTINCT
    region as Region,
    resourcename as FunctionName,
    json_extract_scalar(lambda_json, '$.memorySize') AS memory_size,
    json_extract_scalar(lambda_json, '$.timeout') AS timeout,
    json_extract_scalar(lambda_json, '$.version') AS version
  FROM
    unnested
  WHERE
    lambda_configuration LIKE '%arn:aws:lambda:us-
east-1:111122223333:layer:AnyGovernanceLayer:24%'
```

Here are results from the query:

Query results | Query stats

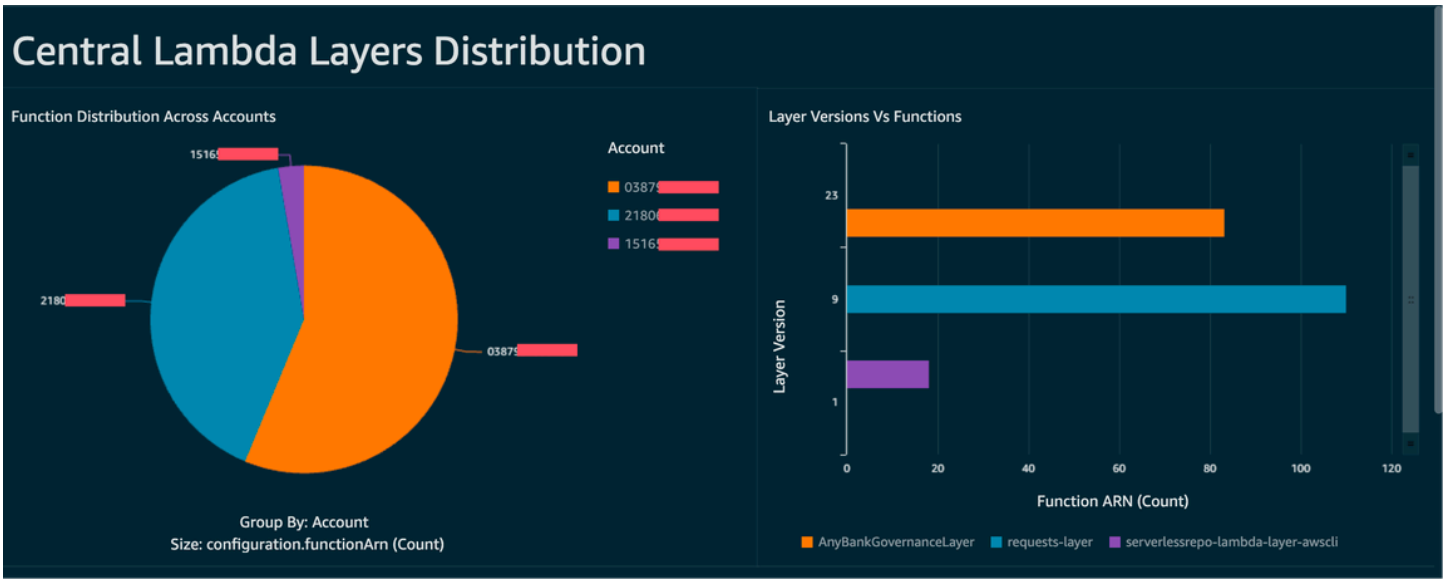
Completed — Time in queue: 127 ms — Run time: 1.803 sec — Data scanned: 239.40 KB

Results (27) — Copy — Download results

Search rows — 1

#	Region	FunctionName	memory_size	timeout	version
1	us-east-1	UpdateUIForPublishEvents	128	18	$LATEST
2	us-east-1	SchedulerCLI-InstanceSchedulerMain	128	300	$LATEST
3	us-east-1	my_functions_function10	128	3	$LATEST
4	us-east-1	lex-web-ui-CognitoIdentityP-CleanStackNameFunction-1TSORSH6L6YXQ	128	300	$LATEST
5	us-east-1	GetLatestArn	128	3	$LATEST
6	us-east-1	aws-python-http-api-project-dev-hello	1024	6	$LATEST
7	us-east-1	cloud9-MyTest-MyTest-688JGPVYP37L	128	15	$LATEST
8	us-east-1	my_functions_function1	128	3	$LATEST
9	us-east-1	my_functions_function25	128	3	$LATEST

With the AWS Config data aggregated across the organization, you can then create a dashboard using Amazon Quick. By importing your Athena results into Quick, you can visualize how well your Lambda functions adhere to the layer version rule. This dashboard can highlight compliant and non-compliant resources, which helps you to determine your enforcement policy, as outlined in the next section. The following image is an example dashboard that reports on the distribution of layer versions applied to functions within the organization.



Phase 3: Implement and enforce

You can now optionally pair your layer version rule that you created in phase 1 with a remediation action via a Systems Manager automation document, which you author as a Python script written with AWS SDK for Python (Boto3). The script calls the UpdateFunctionConfiguration API action for each Lambda function, updating the function configuration with the new layer ARN. Alternatively, you could have the script submit a pull request to the code repository to update the layer ARN. This way future code deployments are also updated with the correct layer ARN.

Lambda code signing with AWS Signer

AWS Signer is a fully managed code-signing service that allows you to validate your code against a digital signature to confirm that code is unaltered and from a trusted publisher. AWS Signer can be used in conjunction with AWS Lambda to verify that functions and layers are unaltered prior to deployment into your AWS environments. This protects your organization from malicious actors who might have gained credentials to create new or update existing functions.

To set up code signing for your Lambda functions, start by creating an S3 bucket with versioning enabled. After that, create a signing profile with AWS Signer, specify Lambda as the platform and then specify a period of days in which the signing profile is valid. Example:

```
Signer:
  Type: AWS::Signer::SigningProfile
  Properties:
    PlatformId: AWSLambda-SHA384-ECDSA
    SignatureValidityPeriod:
      Type: DAYS
      Value: !Ref pValidDays
```

Then use the signing profile and create a signing configuration with Lambda. You have to specify what to do when the signing configuration sees an artifact that does not match a digital signature that it expected: warn (but allow the deployment) or enforce (and block the deployment). The example below is configured to enforce and block deployments.

```
SigningConfig:
  Type: AWS::Lambda::CodeSigningConfig
  Properties:
    AllowedPublishers:
      SigningProfileVersionArns:
        - !GetAtt Signer.ProfileVersionArn
    CodeSigningPolicies:
      UntrustedArtifactOnDeployment: Enforce
```

You now have AWS Signer configured with Lambda to block untrusted deployments. Let's assume you've finished coding a feature request and are now ready to deploy the function. The first step is to zip the code up with the appropriate dependencies and then sign the artifact using the signing profile that you created. You can do this by uploading the zip artifact to the S3 bucket and then starting a signing job.

```
aws signer start-signing-job \
--source 's3={bucketName=your-versioned-bucket,key=your-prefix/your-zip-
artifact.zip,version=QyaJ3c4qa50LXV.9VaZgXHlsGbvCXxpT}' \
--destination 's3={bucketName=your-versioned-bucket,prefix=your-prefix/}' \
--profile-name your-signer-id
```

You get an output as follows, where the `jobId` is the object that is created in the destination bucket and prefix and `jobOwner` is the 12-digit AWS account ID where the job was run.

```
{
    "jobId": "87a3522b-5c0b-4d7d-b4e0-4255a8e05388",
    "jobOwner": "111122223333"
  }
```

And now you can deploy your function using the signed S3 object and the code signing configuration that you created.

```
  Fn:
    Type: AWS::Serverless::Function
    Properties:
      CodeUri: s3://your-versioned-bucket/your-prefix/87a3522b-5c0b-4d7d-
b4e0-4255a8e05388.zip
      Handler: fn.handler
      Role: !GetAtt FnRole.Arn
      CodeSigningConfigArn: !Ref pSigningConfigArn
```

You can alternatively test a function deployment with the original unsigned source zip artifact. The deployment should fail with the following message:

```
Lambda cannot deploy the function. The function or layer might be signed using a
 signature that the client is not configured to accept. Check the provided signature
 for unsigned.
```

If you are building and deploying your functions using the AWS Serverless Application Model (AWS SAM), the package command handles uploading the zip artifact to S3 and also starts the signing job and gets the signed artifact. You can do this with the following command and parameters:

```
sam package -t your-template.yaml \
--output-template-file your-output.yaml \
--s3-bucket your-versioned-bucket \
```

```
--s3-prefix your-prefix \
--signing-profiles your-signer-id
```

AWS Signer helps you verify that zip artifacts that are deployed into your accounts are trusted for deployment. You can include the process above in your CI/CD pipelines and require that all functions have a code signing configuration attached using the techniques outlined in previous topics. By using code signing with your Lambda function deployments, you prevent malicious actors who might have gotten credentials to create or update functions from injecting malicious code in your functions.

Automate security assessments for Lambda with Amazon Inspector

Amazon Inspector is a vulnerability management service that continually scans workloads for known software vulnerabilities and unintended network exposure. Amazon Inspector creates a finding that describes the vulnerability, identifies the affected resource, rates the severity of the vulnerability, and provides remediation guidance.

Amazon Inspector support provides continuous, automated security vulnerability assessments for Lambda functions and layers. Amazon Inspector provides two scan types for Lambda:

- **Lambda standard scanning (default):** Scans application dependencies within a Lambda function and its layers for package vulnerabilities.
- **Lambda code scanning**: Scans the custom application code in your functions and layers for code vulnerabilities. You can either activate Lambda standard scanning or activate Lambda standard scanning together with Lambda code scanning.

To enable Amazon Inspector, navigate to the Amazon Inspector console, expand the **Settings** section, and choose **Account Management**. On the **Accounts** tab, choose **Activate**, and then select one of the scan options.

You can enable Amazon Inspector for multiple accounts and delegate permissions to manage Amazon Inspector for the organization to specific accounts while setting up Amazon Inspector. While enabling, you need to grant Amazon Inspector permissions by creating the role: `AWSServiceRoleForAmazonInspector2`. The Amazon Inspector console allows you to create this role using a one-click option.

For Lambda standard scanning, Amazon Inspector initiates vulnerability scans of Lambda functions in the following situations:

- As soon as Amazon Inspector discovers an existing Lambda function.
- When you deploy a new Lambda function.
- When you deploy an update to the application code or dependencies of an existing Lambda function or its layers.
- Whenever Amazon Inspector adds a new common vulnerabilities and exposures (CVE) item to its database, and that CVE is relevant to your function.

For Lambda code scanning, Amazon Inspector evaluates your Lambda function application code using automated reasoning and machine learning that analyzes your application code for overall security compliance. If Amazon Inspector detects a vulnerability in your Lambda function application code, Amazon Inspector produces a detailed **Code Vulnerability** finding. For a list of possible detections, see the Amazon CodeGuru Detector Library.

To view the findings, go to the Amazon Inspector console. On the **Findings** menu, choose **By Lambda function** to display the security scan results that were performed on Lambda functions.

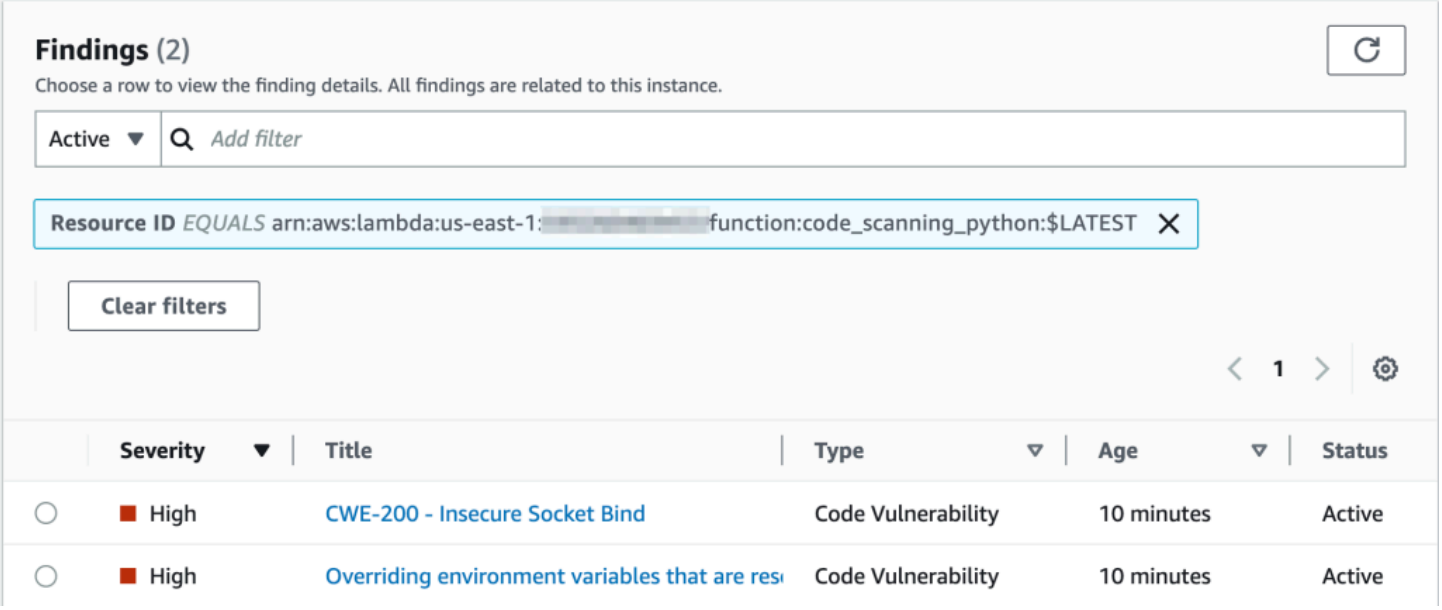
To exclude a Lambda function from standard scanning, tag the function with the following key-value pair:

- `Key:InspectorExclusion`
- `Value:LambdaStandardScanning`

To exclude a Lambda function from code scans, tag the function with the following key-value pair:

- `Key:InspectorCodeExclusion`
- `Value:LambdaCodeScanning`

For example, as shown in following image, Amazon Inspector automatically detects vulnerabilities and categorizes the findings of type **Code Vulnerability**, which indicates that the vulnerability is in the code of the function, and not in one of the code-dependent libraries. You can check these details for a specific function or multiple functions at once.



You can dive further into each of these findings and learn how to remediate the issue.

Overriding environment variables that are reserved by AWS Lambda might lead to unexpected behavior.

Finding ID: arn:aws:inspector2:us-east-1: :finding/

Overriding environment variables that are reserved by AWS Lambda might lead to unexpected behavior or failure of the Lambda function.

Finding overview	
AWS account ID	
Severity	High
Type	Code Vulnerability
Detector name	Override of reserved variable names in a Lambda function
Relevant CWE	--
Rule ID	Rule-434311
Detector tags	#availability, #aws-python-sdk, #aws-lambda, #data-integrity, #maintainability, #security, #security-context, #python
Fix available	Yes
Created at	March 29, 2023 10:08 AM (UTC-04:00)

Vulnerability details	
File path	lambda_function.py

Vulnerability location

```
3  import socket
4
5  def lambda_handler(event, context):
6
7      # print("Scenario 1");
8      os.environ['_HANDLER'] = 'hello'
9      # print("Scenario 1 ends")
10
11     # print("Scenario 2");
12     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
13     s.bind(('',0))
```

Suggested remediation

Your code attempts to override an environment variable that is reserved by the Lambda runtime environment. This can lead to unexpected behavior and might break the execution of your Lambda function.

While working with your Lambda functions, ensure that you comply with the naming conventions for your Lambda functions. For more information, see Working with Lambda environment variables.

You are responsible for the remediation suggestions that you accept. Always review remediation suggestions before accepting them. You might need to make edits to remediation suggestions to ensure that your code does what you intended.

Implement observability for Lambda security and compliance

AWS Config is a useful tool to find and fix non-compliant AWS Serverless resources. Every change you make to your serverless resources is recorded in AWS Config. Additionally, AWS Config allows you to store configuration snapshot data on S3. You can use Amazon Athena and Amazon Quick to make dashboards and see AWS Config data. In Detect non-compliant Lambda deployments and configurations with AWS Config, we discussed how we can visualize a certain configuration like Lambda layers. This topic expands on these concepts.

Visibility into Lambda configurations

You can use queries to pull important configurations like AWS account ID, Region, AWS X-Ray tracing configuration, VPC configuration, memory size, runtime, and tags. Here is a sample query you can use to pull this information from Athena:

```
WITH unnested AS (
    SELECT
      item.awsaccountid AS account_id,
      item.awsregion AS region,
      item.configuration AS lambda_configuration,
      item.resourceid AS resourceid,
      item.resourcename AS resourcename,
      item.configuration AS configuration,
      json_parse(item.configuration) AS lambda_json
    FROM
      default.aws_config_configuration_snapshot,
      UNNEST(configurationitems) as t(item)
    WHERE
      "dt" = 'latest'
      AND item.resourcetype = 'AWS::Lambda::Function'
  )

  SELECT DISTINCT
    account_id,
    tags,
    region as Region,
    resourcename as FunctionName,
    json_extract_scalar(lambda_json, '$.memorySize') AS memory_size,
    json_extract_scalar(lambda_json, '$.timeout') AS timeout,
    json_extract_scalar(lambda_json, '$.runtime') AS version
    json_extract_scalar(lambda_json, '$.vpcConfig.SubnetIds') AS vpcConfig
    json_extract_scalar(lambda_json, '$.tracingConfig.mode') AS tracingConfig
```

```
  FROM
    unnested
```

You can use the query to build an Quick dashboard and visualize the data. To aggregate AWS resource configuration data, create tables in Athena, and build Quick dashboards on the data from Athena, see Visualizing AWS Config data using Athena and Amazon Quick on the AWS Cloud Operations and Management blog. Notably, this query also retrieves tag information for the functions. This allows for deeper insights into your workloads and environments, especially if you employ custom tags.

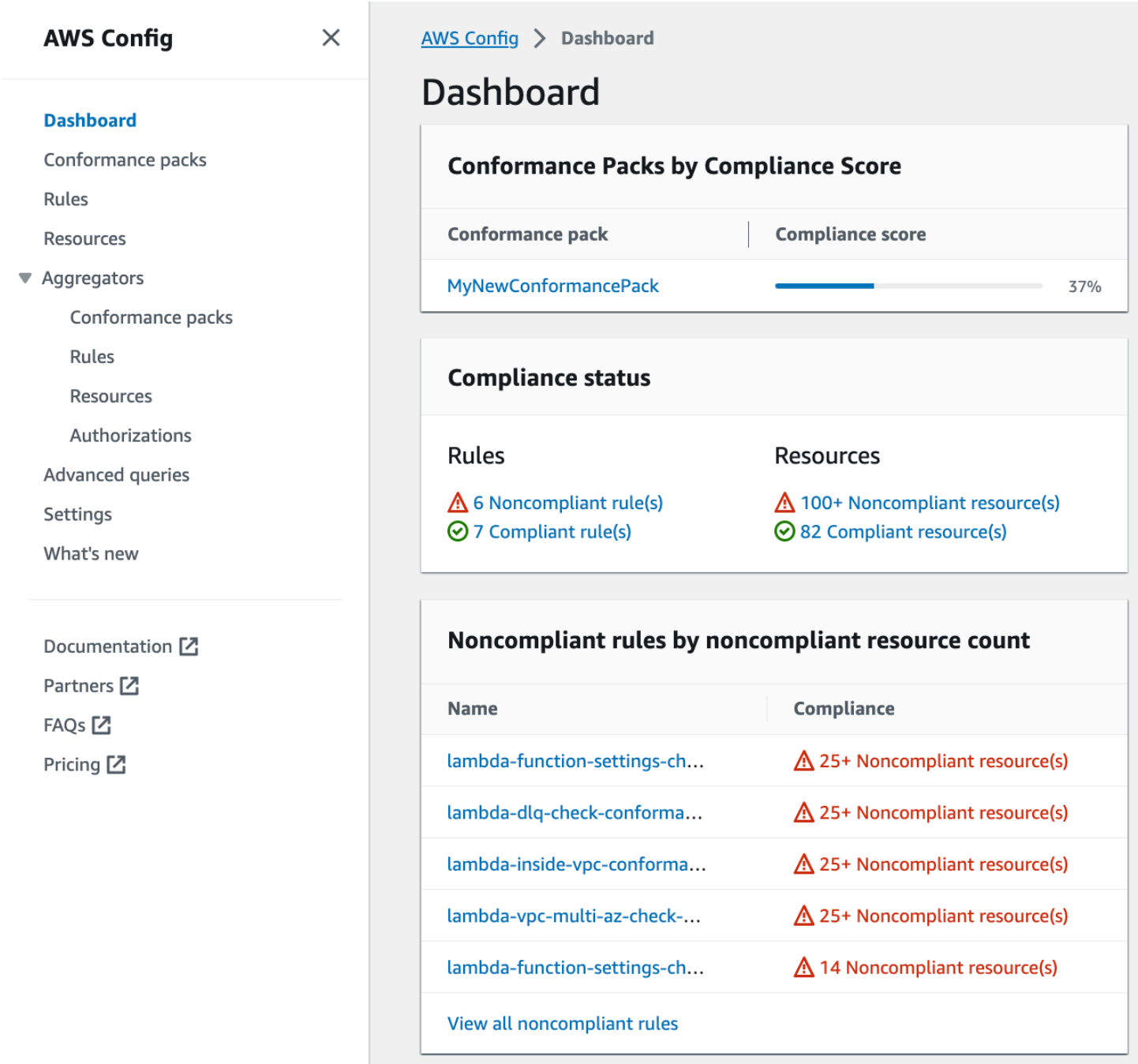
For more information on actions that you can take, see the Addressing the observability findings section later in this topic.

Visibility into Lambda compliance

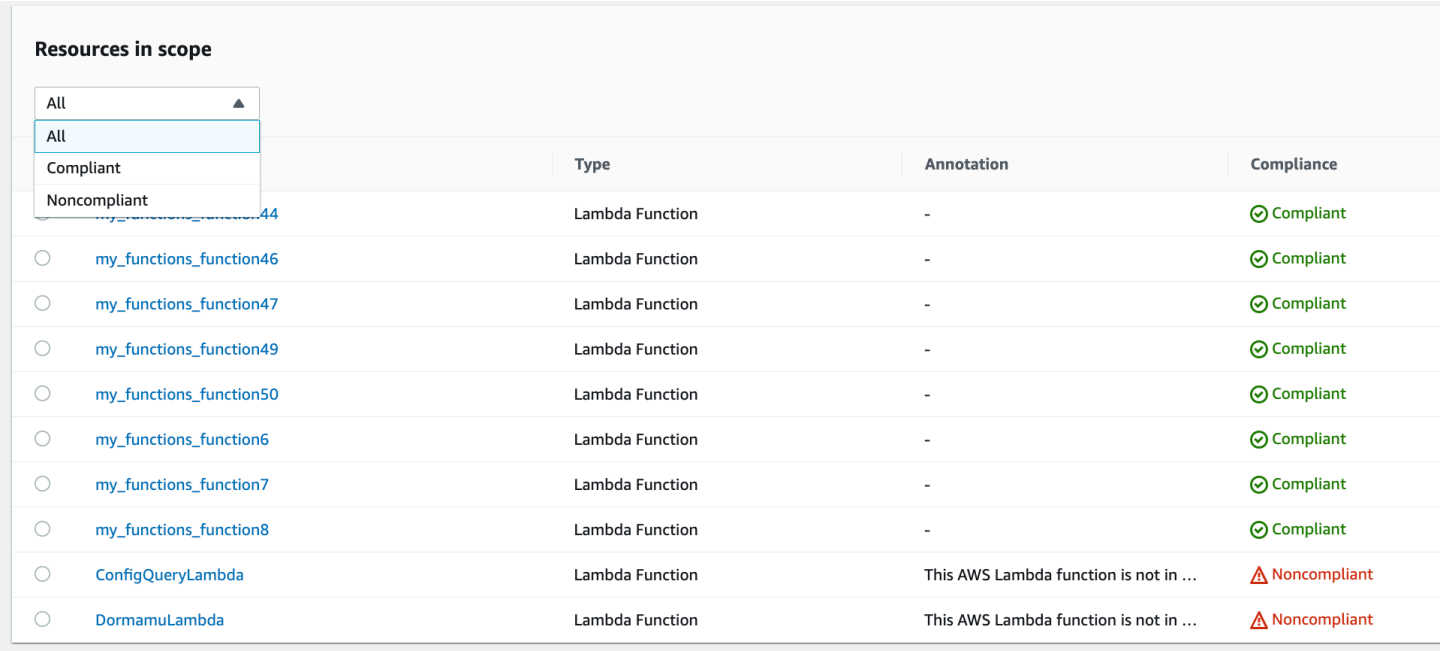
With the data generated by AWS Config, you can create organization-level dashboards to monitor compliance. This allows for consistent tracking and monitoring of:

- Compliance packs by compliance score
- Rules by non-compliant resources

- Compliance status

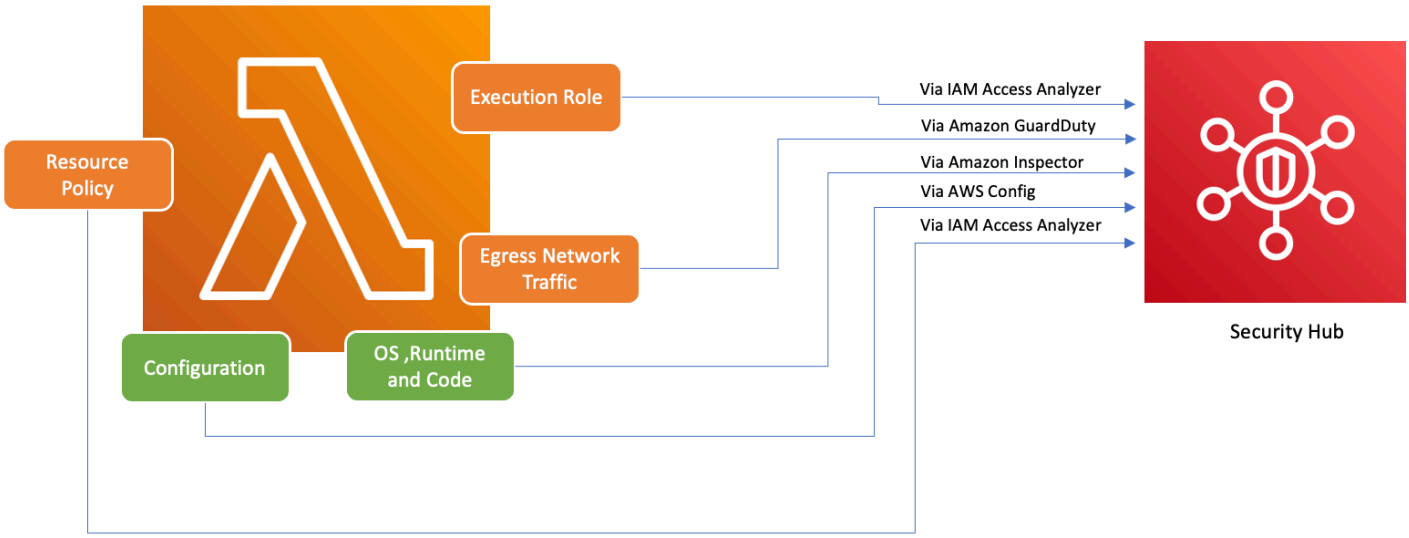


Check each rule to identify non-compliant resources for that rule. For example, if your organization mandates that all Lambda functions must be associated with a VPC and if you have deployed an AWS Config rule to identify compliance, you can select the `lambda-inside-vpc` rule in the list above.



For more information on actions that you can take, see the Addressing the observability findings section below.

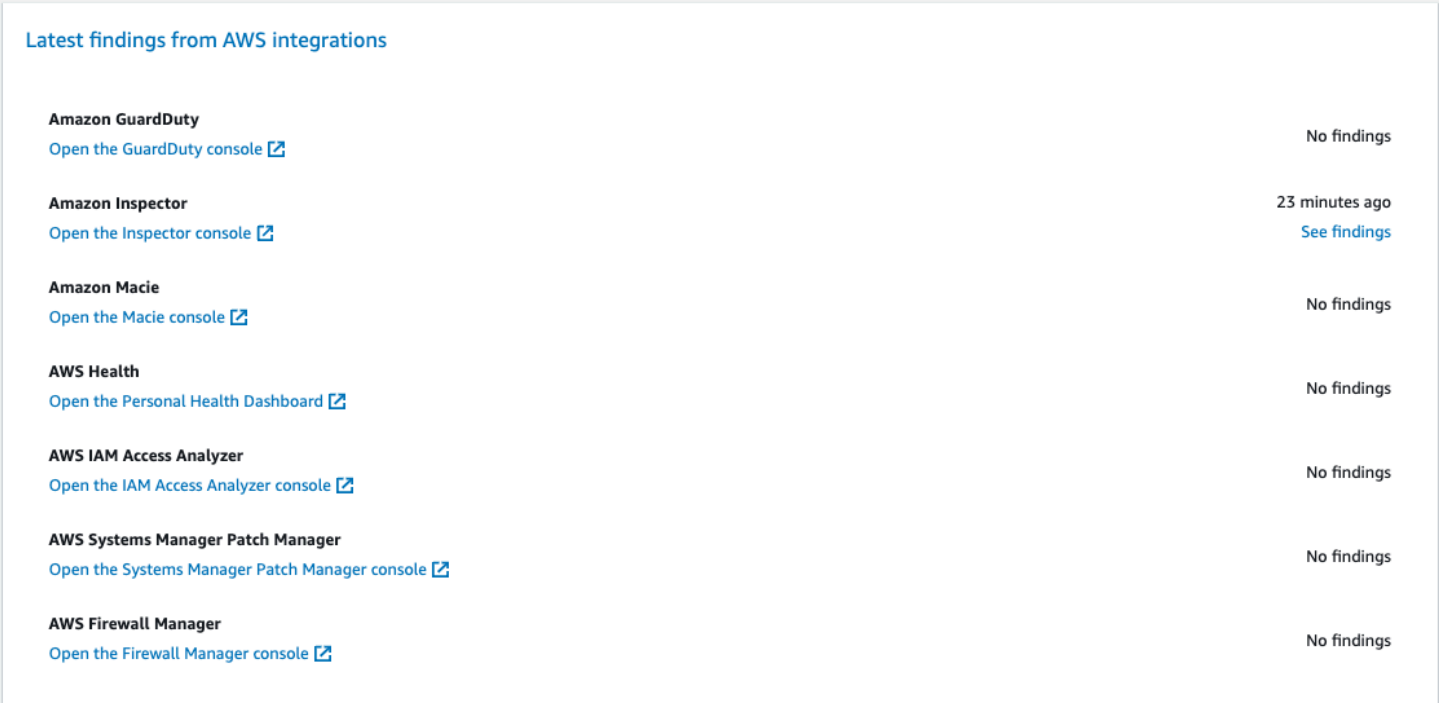
Visibility into Lambda function boundaries using Security Hub CSPM



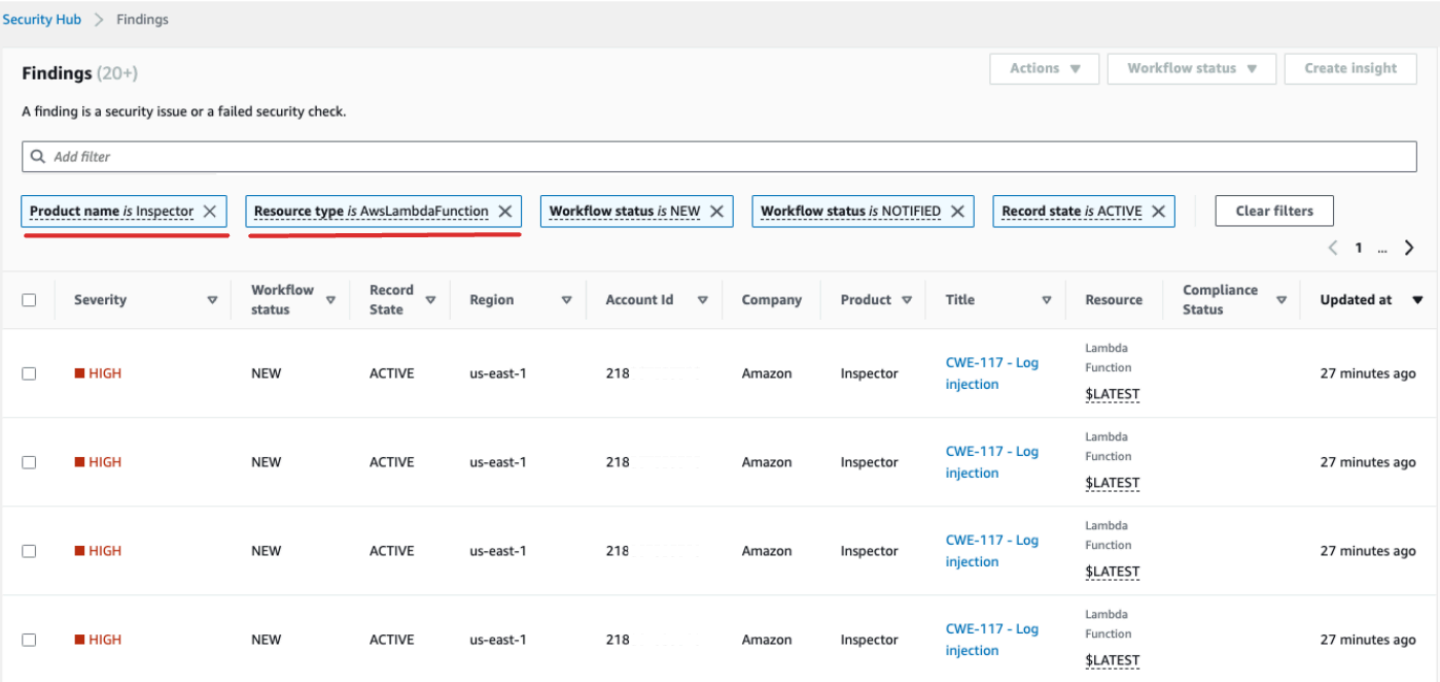
To ensure that AWS services including Lambda are used securely, AWS introduced the Foundational Security Best Practices v1.0.0. This set of best practices provides clear guidelines for securing resources and data in the AWS environment, emphasizing the importance of maintaining a strong security posture. The AWS Security Hub CSPM complements this by offering a unified security and compliance center. It aggregates, organizes, and prioritizes security findings from multiple

AWS services like Amazon Inspector, AWS Identity and Access Management Access Analyzer, and Amazon GuardDuty.

If you have Security Hub CSPM, Amazon Inspector, IAM Access Analyzer, and GuardDuty enabled within your AWS organization, Security Hub CSPM automatically aggregates findings from these services. For instance, let's consider Amazon Inspector. Using Security Hub CSPM, you can efficiently identify code and package vulnerabilities in Lambda functions. In the Security Hub CSPM console, navigate to the bottom section labeled **Latest findings from AWS integrations**. Here, you can view and analyze findings sourced from various integrated AWS services.



To see details, choose the **See findings** link in the second column. This displays a list of findings filtered by product, such as Amazon Inspector. To limit your search to Lambda functions, set `ResourceType` to `AwsLambdaFunction`. This displays findings from Amazon Inspector related to Lambda functions.



For GuardDuty, you can identify suspicious network traffic patterns. Such anomalies might suggest the existence of potentially malicious code within your Lambda function.

With IAM Access Analyzer, you can check policies, especially those with condition statements that grant function access to external entities. Moreover, IAM Access Analyzer evaluates permissions set when using the AddPermission operation in the Lambda API alongside an `EventSourceToken`.

Addressing the observability findings

Given the wide-ranging configurations possible for Lambda functions and their distinct requirements, a standardized automation solution for remediation might not suit every situation. Additionally, changes are implemented differently across various environments. If you encounter any configuration that seems non-compliant, consider the following guidelines:

1. **Tagging strategy**

   We recommend implementing a comprehensive tagging strategy. Each Lambda function should be tagged with key information such as:

   - **Owner:** The person or team responsible for the function.
   - **Environment:** Production, staging, development, or sandbox.
   - **Application:** The broader context to which this function belongs, if applicable.

2. **Owner outreach**

Instead of automating the breaking changes (like VPC configuration adjustment), proactively contact the owners of non-compliant functions (identified by the owner tag) providing them sufficient time to either:

- Adjust non-compliant configurations on Lambda functions.
- Provide an explanation and request an exception, or refine the compliance standards.

3. **Maintain a configuration management database (CMDB)**

   While tags can provide immediate context, maintaining a centralized CMDB can provide deeper insights. It can hold more granular information about each Lambda function, its dependencies, and other critical metadata. A CMDB is an invaluable resource for auditing, compliance checks, and identifying function owners.

As the landscape of serverless infrastructure continually evolves, it's essential to adopt a proactive stance towards monitoring. With tools like AWS Config, Security Hub CSPM, and Amazon Inspector, potential anomalies or non-compliant configurations can be swiftly identified. However, tools alone cannot ensure total compliance or optimal configurations. It's crucial to pair these tools with well-documented processes and best practices.

- **Feedback loop:** Once remediation steps are undertaken, ensure there's a feedback loop. This means periodically revisiting non-compliant resources to confirm if they've been updated or are still running with the same issues.
- **Documentation:** Always document the observations, actions taken, and any exceptions granted. Proper documentation not only helps during audits but also aids in enhancing the process for better compliance and security in the future.
- **Training and awareness:** Ensure that all stakeholders, especially Lambda function owners, are regularly trained and made aware of best practices, organizational policies, and compliance mandates. Regular workshops, webinars, or training sessions can go a long way in ensuring everyone is on the same page when it comes to security and compliance.

In conclusion, while tools and technologies provide robust capabilities to detect and flag potential issues, the human element—understanding, communication, training, and documentation—remains pivotal. Together, they form a potent combination to ensure that your Lambda functions and broader infrastructure remain compliant, secure, and optimized for your business needs.